

2026년 07월 8일 학습 발표

트랜스포머 인코더 기반 베어링 이상진단

Transformer Encoder-Based Bearing Fault Diagnosis

- Presenter: Gyeong Ho Min

1. 데이터셋 소개

1.1 CWRU 데이터셋 개요

1.2 파일 구성

1.3 부하 조건 및 파일 내부 구조

1.4 샘플 수

2. 데이터 전처리

2.1 슬라이딩 윈도우

2.2 정규화 (StandardScaler)

2.3 정규화 결과

2.4 텐서 변환

2.5 데이터 분할 및 DataLoader

3. 모델 학습

3.1 학습 흐름 및 원리

3.2 손실함수

3.3 Cross Entropy Error

3.4 학습 설정

4. 모델 구조 (Transformer Encoder)

4.1 전체 아키텍처

4.2 Transformer layer 코드

5. 실험 결과

5.1 학습 곡선 (Loss/ACC)

5.2 Confusion Matrix

5.3 LSTM vs TST 비교

6. 결론

1. 데이터셋 소개

1.1 CWRU 데이터셋 개요

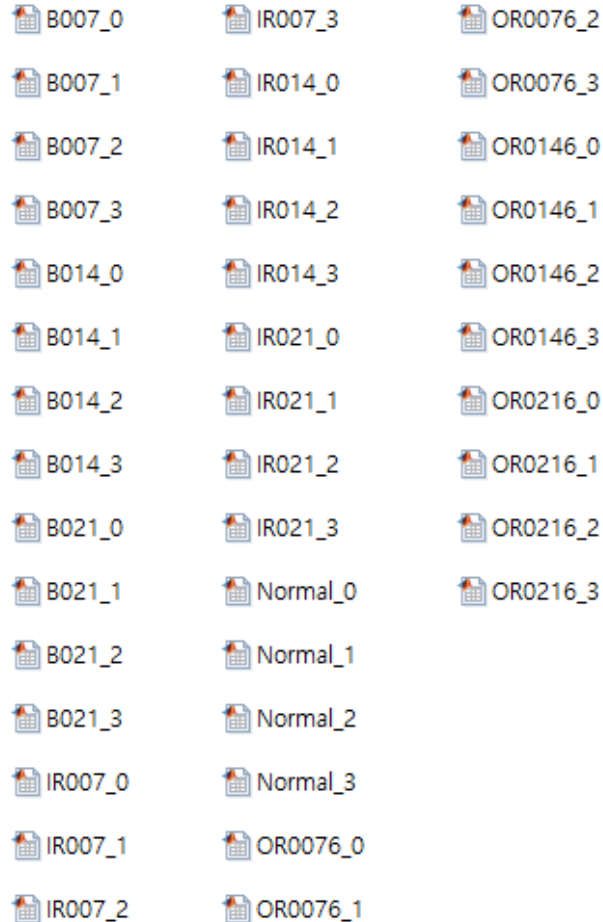
- CWRU(Case Western Reserve University) 데이터셋은 베어링 고장 분류 알고리즘의 벤치마크 데이터셋으로 널리 사용됨
- 실제 실험 환경에서 베어링에 인위적으로 결함을 생성하여 수집한 진동 신호 데이터로 구성

✓ 클래스 구성

클래스	설명
Normal (N)	결함 없는 정상 베어링
Ball (B)	볼에 결함이 있는 베어링
Inner Race (IR)	내륜에 결함이 있는 베어링
Outer Race (OR)	외륜에 결함이 있는 베어링

1.2 파일 구성

데이터 셋은 총 40개의 .mat 파일로 구성되어 있으며 파일 명은 아래 규칙을 따름.



클래스	결함 크기	파일 수
Normal	-	4개 (Normal_0~3)
Ball	0.007 / 0.014 / 0.021인치	12개
Inner Race	0.007 / 0.014 / 0.021인치	12개
Outer Race	0.007 / 0.014 / 0.021인치	12개
합계		40개

결함은 전기방전가공(EDM) 방식으로 직경 0.007~0.021인치 크기로 인위적으로 생성
(EDM은 전기 불꽃으로 금속을 정밀하게 깎는 기술로, 0.001인치 단위의 정밀한 결함 생성이 가능)

```
import os

path = r"C:\Users\rudgh\Downloads\bearing\cwru"

for f in os.listdir(path):
    print(f)

[1]
```

1.3 부하 조건 및 파일 내부 구조

부하 조건

- ✓ 각 파일은 서로 다른 부하 조건에서 측정

파일 번호	부하	회전속도
_0	0 hp	1797 RPM
_1	1 hp	1772 RPM
_2	2 hp	1750 RPM
_3	3 hp	1730 RPM

파일 내부 구조

- ✓ 각 .mat 파일 내부에는 아래 변수들을 포함

변수명	설명	포함 여부
X000_DE_time	Drive End 진동 신호	항상 포함
X000_FE_time	Fan End 진동 신호	대부분 포함
X000_BA_time	Base 진동 신호	일부 포함
X000 RPM	회전속도	일부 포함

```
import scipy.io

mat = scipy.io.loadmat(r"C:\Users\rudgh\Downloads\bearing\cwru\Normal_0.mat")
print(mat.keys())

[4]

import numpy as np

data = mat['X097_DE_time']
print(data.shape)
print(data[:5])

[5]
```

본 연구에서는 결함 신호가 가장 뚜렷하게 나타나는 Drive End(DE) 채널만 사용.

1.4 샘플 수

- 각 파일의 샘플 수는 파일마다 다르다. Kaggle 데이터셋 설명에는 약 12만~24만 개라고 기재되어 있으나, 실제로 직접 확인한 결과 Normal 파일의 경우 더 많은 샘플이 포함.

파일	샘플링 주파수	샘플 수
Normal_0.mat	12 kHz	243,938개
Normal_1~3.mat	48 kHz	483,903개
B007_0~3.mat	12 kHz	약 121,000~122,000개
IR, OR 파일	12 kHz	약 120,000~122,000개

2. 데이터 전처리

2.1 슬라이딩 윈도우

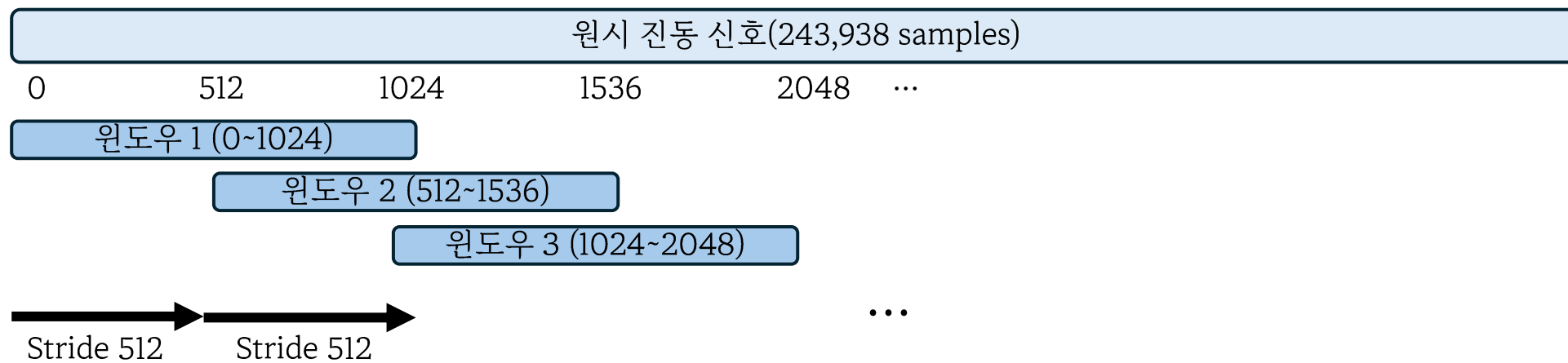
- 각 .mat 파일의 진동 신호는 약 12만~48만 개의 샘플로 구성되어 있음
- 윈도우 기법을 적용하여 긴 신호를 일정 크기의 조각으로 잘라 각각 하나의 샘플로 만들

윈도우 크기는 1024샘플로 설정

- 윈도우 1개에 베어링이 약 2.5바퀴 회전하는 동안의 신호가 담기며, 결함 패턴이 최소 2번 이상 나타나 학습에 충분한 정보를 포함.

슬라이딩 윈도우 동작 방식

이동 간격(Stride)은 512샘플로 윈도우는 50% 겹치도록 설정



50% 겹침을 적용한 이유는 경계 부분의 패턴을 놓치지 않고 데이터 수를 늘려 학습 효과를 향상시키기 위함.

2.1 슬라이딩 윈도우

최종 데이터 구성

클래스	샘플 수
Normal	3,310개
Ball	2,840개
Inner Race	2,838개
Outer Race	2,844개
합계	11,832개

```
import os
import numpy as np
import scipy.io

path = r"C:\Users\rudgh\Downloads\bearing\cwru"

# 클래스 매핑
label_map = {
    'Normal': 0,
    'B': 1,
    'IR': 2,
    'OR': 3
}

def get_label(filename):
    for key in label_map:
        if filename.startswith(key):
            return label_map[key]
    return None

# DE 채널 키 찾기
def get_de_key(mat):
    for k in mat.keys():
        if 'DE_time' in k:
            return k
    return None
```

```
# 윈도우 슬라이싱
WINDOW_SIZE = 1024
STRIDE = 512

X, y = [], []

for fname in sorted(os.listdir(path)):
    if not fname.endswith('.mat'):
        continue
    label = get_label(fname)
    if label is None:
        continue

    mat = scipy.io.loadmat(os.path.join(path, fname))
    de_key = get_de_key(mat)
    if de_key is None:
        continue

    signal = mat[de_key].flatten()

    for start in range(0, len(signal) - WINDOW_SIZE, STRIDE):
        #           ↑ 0부터 ↑ 마지막 유효 시작점까지 ↑ 512씩
        window = signal[start:start + WINDOW_SIZE]
        X.append(window)
        y.append(label)

X = np.array(X)
y = np.array(y)

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"클래스 분포: {np.bincount(y)}")
```

2.2 정규화 (Z-score)

- CWRU 데이터셋의 각 클래스는 진동 신호의 크기가 서로 다름

클래스	신호 범위	표준편차
Normal	-0.2 ~ 0.2	작음
Ball	-0.4 ~ 0.4	중간
IR	-1.0 ~ 1.5	큼
OR	-2.5 ~ 2.5	제일 큼

정규화 방법

본 연구에서는 StandardScaler를 사용하여 정규화를 적용

$$\text{정규화값} = (\text{원래값} - \text{평균}) / \text{표준편차}$$

- ① 평균을 빼면 데이터가 0 중심으로 이동하여 평균은 0
- ② 표준편차로 나누면 퍼진 정도가 1로 통일되어 표준편차는 1

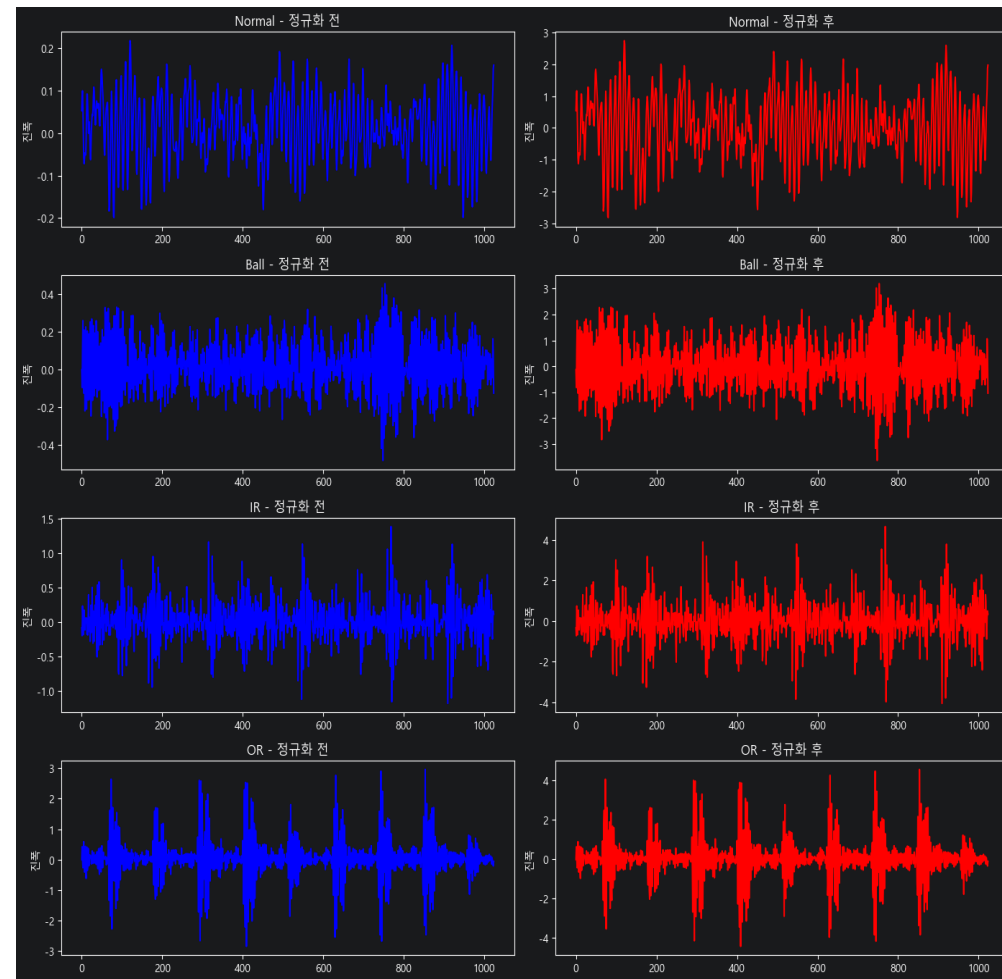
2.3 정규화 결과

정규화 결과

클래스	정규화 전 범위	정규화 후 범위	표준편차
Normal	-0.2 ~ 0.2	-3 ~ 3	1
Ball	-0.4 ~ 0.4	-3 ~ 3	1
IR	-1.0 ~ 1.5	-4 ~ 4	1
OR	-2.5 ~ 2.5	-4 ~ 4	1

정규화의 효과

- 모든 클래스 표준편차 1로 통일되어 특정 클래스에 치우치지 않음
- 패턴 학습: 값의 크기가 아닌 진동 패턴(모양)으로 클래스 구분
- 학습 안정화: 퍼진 정도가 통일되어 gradient가 안정적



2.4 텐서 변환

텐서 변환 이유: 딥러닝 학습을 위해 numpy 배열을 PyTorch 텐서로 변환

	numpy 배열	PyTorch 텐서
사용처	전처리 (슬라이딩 윈도우, 정규화)	딥러닝 학습
GPU 사용	불가능	가능
gradient 계산	불가능	가능

```
# Tensor 변환 - TST 입력: (batch, seq_len, features)
X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
            .unsqueeze(-1) # (11832, 1024, 1)
y_tensor = torch.tensor(y, dtype=torch.long)
```

2.5 데이터 분할 및 Data Loader

Train/Val/Test 분할

전체 11,832개 윈도우를 3개로 분할

데이터셋	개수	비율	역할
train_ds	8,282개	70%	모델 학습용
val_ds	1,774개	15%	학습 중 성능 확인용
test_ds	1,776개	15%	최종 평가용

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset, random_split
from sklearn.preprocessing import StandardScaler

# 정규화
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # (11832, 1024)

# Tensor 변환 - TST 입력: (batch, seq_len, features)
X_tensor = torch.tensor(X_scaled, dtype=torch.float32).unsqueeze(-1) # (11832, 1024, 1)
y_tensor = torch.tensor(y, dtype=torch.long)

# Train/Val/Test 분할 (7:1.5:1.5)
dataset = TensorDataset(X_tensor, y_tensor)
n = len(dataset)
n_train = int(n * 0.7)
n_val = int(n * 0.15)
n_test = n - n_train - n_val

train_ds, val_ds, test_ds = random_split(dataset, [n_train, n_val, n_test])

train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=64)
test_loader = DataLoader(test_ds, batch_size=64)

print(f"Train: {n_train}, Val: {n_val}, Test: {n_test}")
```

→무작위 분할로 4개 클래스가 골고루 포함되어 편향 방지

DataLoader

- batch_size=64 : 64개씩 묶어서 모델에 입력
- shuffle=True : 매 epoch마다 순서를 섞어 편향 방지

3. 모델 학습

3.1 학습 원리

① 순전파 (Forward)

`Pred = model (Xb)`

→ 입력이 모델을 통과해서 예측 값 생성

② Loss 계산

`Loss = criterion(pred, yb)`

→ CrossEntropyLoss로 예측이 얼마나 틀렸는지 계산

③ 역전파 (Backward)

`loss.backward()`

→ gradient 계산 (각 파라미터가 loss에 얼마나 영향 줬는지)

④ 파라미터 업데이트 (Gradient Descent)

`optimizer.step()`

→ Adam optimizer가 gradient 방향으로 가중치 조금씩 수정)

```
def forward(self, x):  
    # x: (batch, seq_len, 1)  
    x = self.input_proj(x)           # (batch, seq_len, d_model)  
    x = x + self.pos_embedding       # positional encoding 추가  
    x = self.transformer(x)          # (batch, seq_len, d_model)  
    x = x.mean(dim=1)                # Global Average Pooling  
    return self.classifier(x)
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)  
criterion = nn.CrossEntropyLoss()  
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
```

```
for epoch in range(EPOCHS):  
    # Train  
    model.train()  
    train_loss, train_correct = 0, 0  
    for xb, yb in train_loader:  
        xb, yb = xb.to(device), yb.to(device)  
        optimizer.zero_grad()  
        pred = model(xb)  
        loss = criterion(pred, yb)  
        loss.backward()  
        optimizer.step()  
        train_loss += loss.item()  
        train_correct += (pred.argmax(1) == yb).sum().item()
```

3.2 손실함수

기능: 목표출력에서 실제출력 간의 차이를 계량화하기 위한 함수.

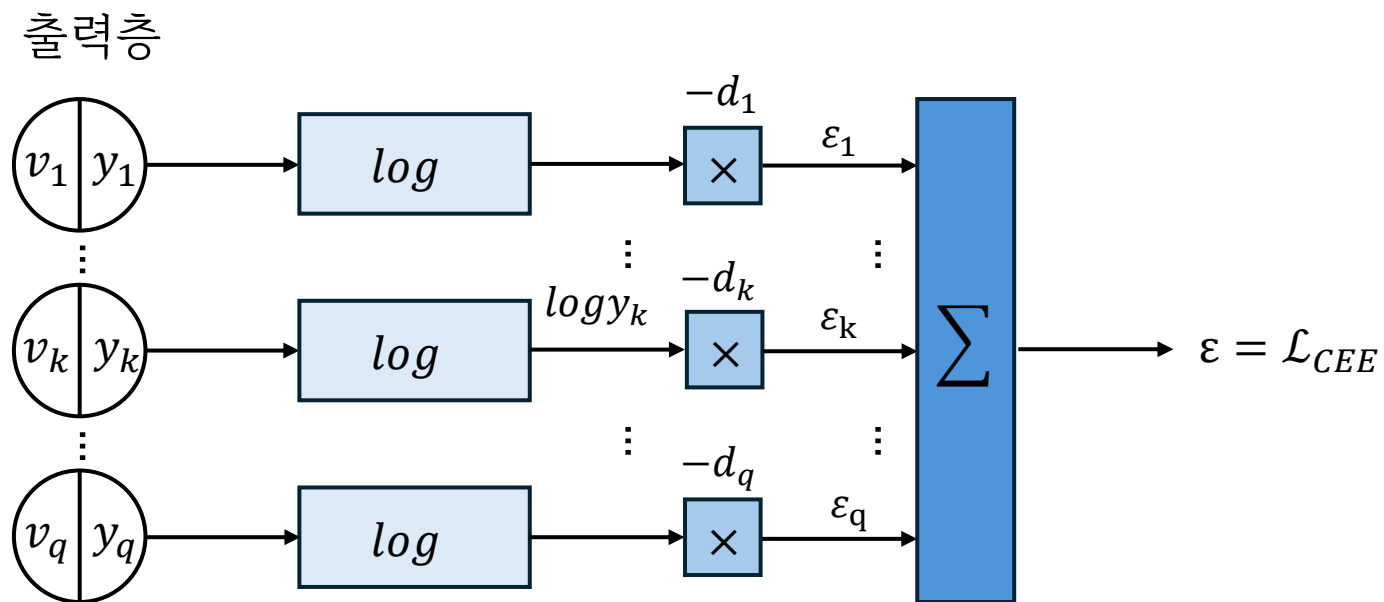
계량화 값 $\rightarrow$ 손실(loss)

용도

- ✓ 손실함수는 추론 오차를 계량화한 수단이며, 추론 오차의 크기는 현재 가중치가 얼마나 적절한지를 나타내는 척도.
- ✓ 손실함수는 가중치 갱신량을 계산하는 데에 사용하며, 갱신량의 크기는 수렴속도에 결정적인 영향을 미치므로 그 크기의 결정에 매우 중요한 역할을 수행
- ✓ 손실함수는 학습 종료 여부를 결정하는 데에 사용되며, 매 epoch 종료 후 전체 훈련 데이터에 대한 평균 손실값을 계산하여 기준값 이하로 수렴하면 학습을 종료

3.3 Cross Entropy Error

Cross Entropy Error: 다중분류 작업에 주로 많이 사용되는 손실함수



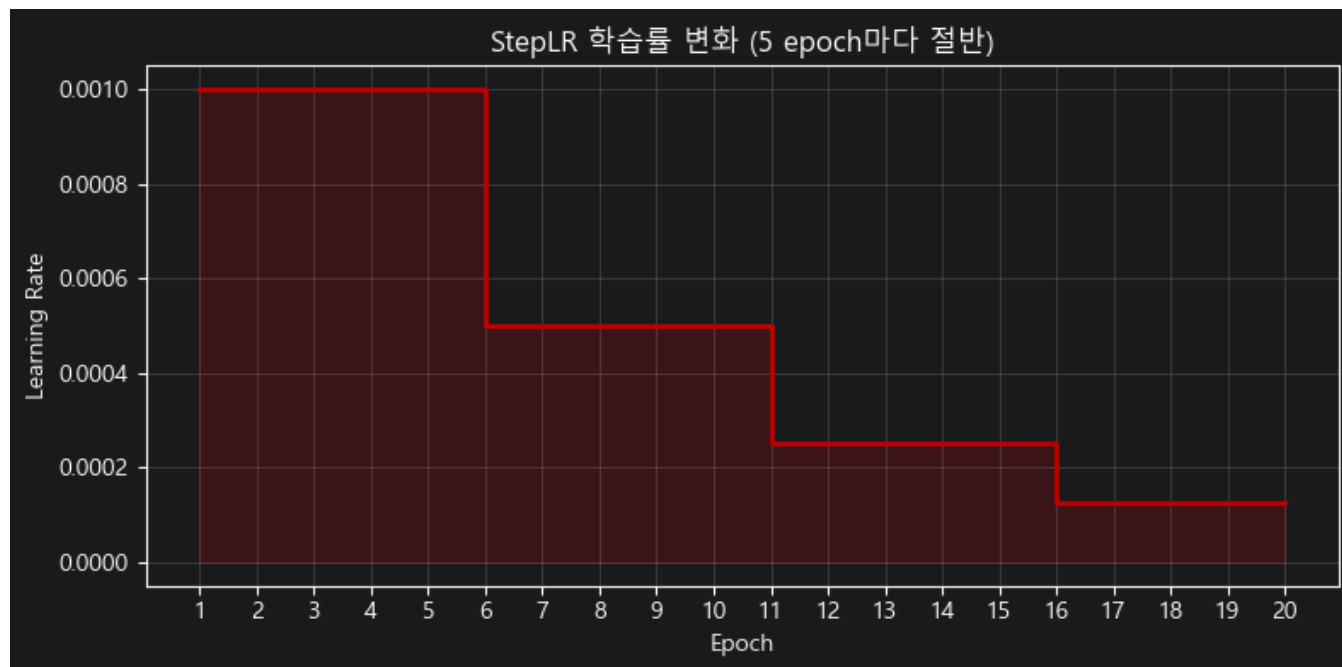
v 는 Activation Function을 거치기 전, 출력층 노드의 가중합 벡터: $v = [v_1, \dots, v_q]^T$

입력샘플에 대한 추론결과인 출력 벡터: $y = [y_1, \dots, y_q]^T$

실제 정답인 목표출력 벡터 (One-hot code): $d = [d_1, \dots, d_q]^T$

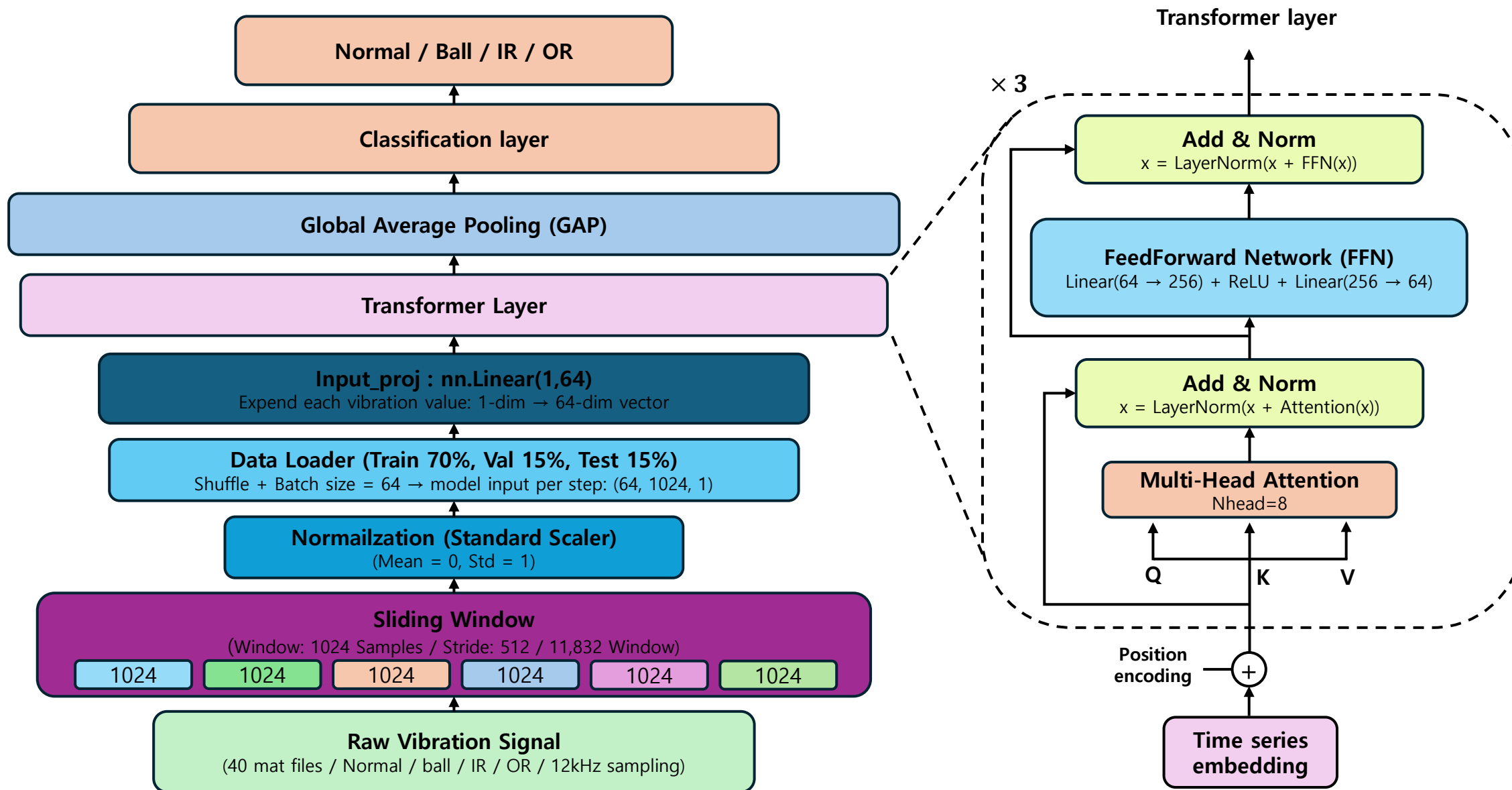
3.4 학습 설정

설정	값	역할
Optimizer	Adam (lr=0.001)	가중치 업데이트
Loss	CrossEntropyLoss	다중분류 손실 함수
Scheduler	StepLR(step=5, $\gamma=0.5$)	5 epoch마다 학습률 절반
Epochs	20	전체 반복 횟수
Batch Size	64	한 번에 학습하는 샘플 수

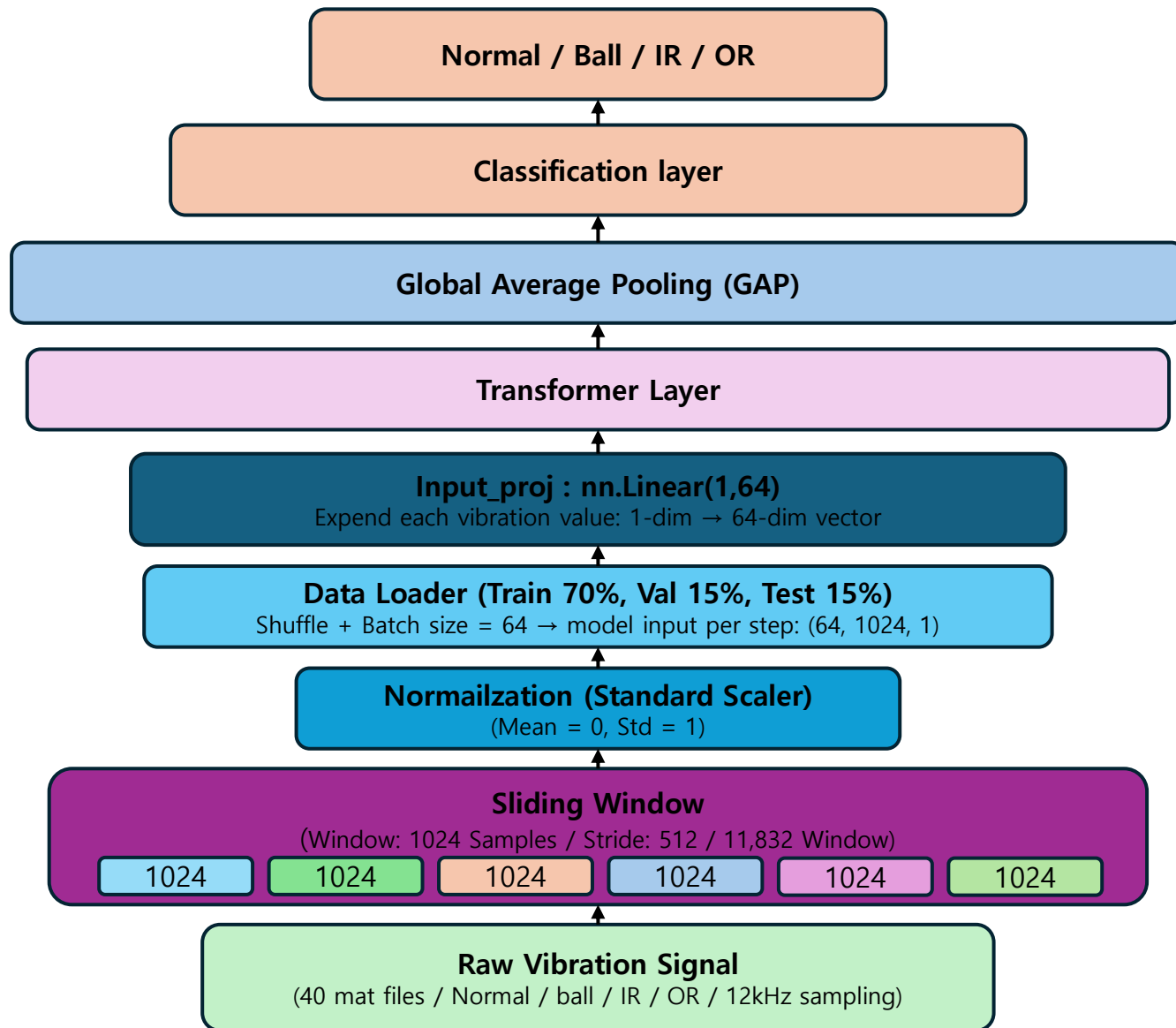


4. 모델 구조 (Transformer Encoder)

4.1 전체 아키텍처



4.2 Transformer Layer 코드



```
class TST(nn.Module):
    def __init__(self, seq_len=1024, d_model=64, nhead=8, num_layers=3, num_classes=4, dropout=0.1):
        super().__init__()

        # 입력 임베딩
        self.input_proj = nn.Linear(1, d_model)

        # Positional Encoding
        self.pos_embedding = nn.Parameter(torch.zeros(1, seq_len, d_model))

        # Transformer Encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=256,
            dropout=dropout,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        # 분류 헤드
        self.classifier = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, num_classes)
        )

    def forward(self, x):
        # x: (batch, seq_len, 1)
        x = self.input_proj(x) # (batch, seq_len, d_model)
        x = x + self.pos_embedding # positional encoding 추가
        x = self.transformer(x) # (batch, seq_len, d_model)
        x = x.mean(dim=1) # Global Average Pooling
        return self.classifier(x)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

model = TST().to(device)
print(model)
```

5. 실험결과

5.1 학습 곡선

```
Epoch 01 | Train Loss: 0.3580 Acc: 0.8690 | Val Loss: 0.1407 Acc: 0.9487
Epoch 02 | Train Loss: 0.1678 Acc: 0.9447 | Val Loss: 0.1825 Acc: 0.9278
Epoch 03 | Train Loss: 0.1555 Acc: 0.9465 | Val Loss: 0.1264 Acc: 0.9594
Epoch 04 | Train Loss: 0.1392 Acc: 0.9544 | Val Loss: 0.1300 Acc: 0.9476
Epoch 05 | Train Loss: 0.1401 Acc: 0.9513 | Val Loss: 0.1192 Acc: 0.9594
Epoch 06 | Train Loss: 0.1099 Acc: 0.9632 | Val Loss: 0.1073 Acc: 0.9600
Epoch 07 | Train Loss: 0.0936 Acc: 0.9697 | Val Loss: 0.0989 Acc: 0.9684
Epoch 08 | Train Loss: 0.0870 Acc: 0.9728 | Val Loss: 0.1219 Acc: 0.9549
Epoch 09 | Train Loss: 0.0745 Acc: 0.9763 | Val Loss: 0.0986 Acc: 0.9645
Epoch 10 | Train Loss: 0.0914 Acc: 0.9714 | Val Loss: 0.0794 Acc: 0.9775
Epoch 11 | Train Loss: 0.0573 Acc: 0.9843 | Val Loss: 0.1105 Acc: 0.9611
Epoch 12 | Train Loss: 0.0563 Acc: 0.9847 | Val Loss: 0.1142 Acc: 0.9622
Epoch 13 | Train Loss: 0.0510 Acc: 0.9854 | Val Loss: 0.1727 Acc: 0.9538
Epoch 14 | Train Loss: 0.0507 Acc: 0.9847 | Val Loss: 0.1231 Acc: 0.9583
Epoch 15 | Train Loss: 0.0495 Acc: 0.9859 | Val Loss: 0.0894 Acc: 0.9690
Epoch 16 | Train Loss: 0.0392 Acc: 0.9887 | Val Loss: 0.0952 Acc: 0.9684
Epoch 17 | Train Loss: 0.0361 Acc: 0.9901 | Val Loss: 0.0983 Acc: 0.9718
Epoch 18 | Train Loss: 0.0372 Acc: 0.9887 | Val Loss: 0.1067 Acc: 0.9662
Epoch 19 | Train Loss: 0.0324 Acc: 0.9905 | Val Loss: 0.0945 Acc: 0.9696
Epoch 20 | Train Loss: 0.0316 Acc: 0.9911 | Val Loss: 0.1202 Acc: 0.9651
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

EPOCHS = 20

for epoch in range(EPOCHS):
    # Train
    model.train()
    train_loss, train_correct = 0, 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
        train_correct += (pred.argmax(1) == yb).sum().item()

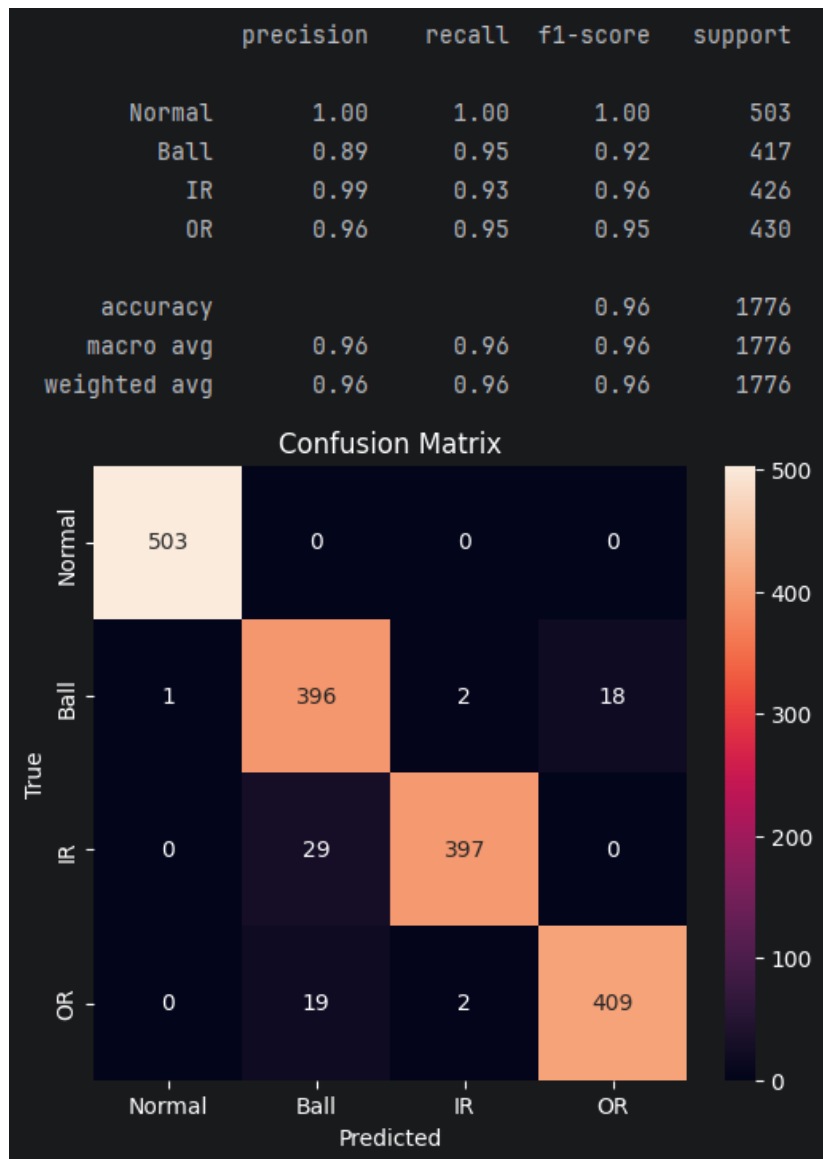
    # Val
    model.eval()
    val_loss, val_correct = 0, 0
    with torch.no_grad():
        for xb, yb in val_loader:
            xb, yb = xb.to(device), yb.to(device)
            pred = model(xb)
            val_loss += criterion(pred, yb).item()
            val_correct += (pred.argmax(1) == yb).sum().item()

    scheduler.step()

    print(f"Epoch {epoch+1:02d} | "
          f"Train Loss: {train_loss/len(train_loader):.4f} Acc: {train_correct/n_train:.4f} | "
          f"Val Loss: {val_loss/len(val_loader):.4f} Acc: {val_correct/n_val:.4f}")
```

[11]

5.2 Confusion Matrix



전체 성능

- ✓ Accuracy 96% / Macro F1 0.96 / Weighted F1 0.96
- ✓ 세 지표 일치 → 균형잡힌 데이터, 편향 없는 학습

클래스별 F1

- ✓ Normal 1.00 / Ball 0.92 / IR 0.96 / OR 0.95

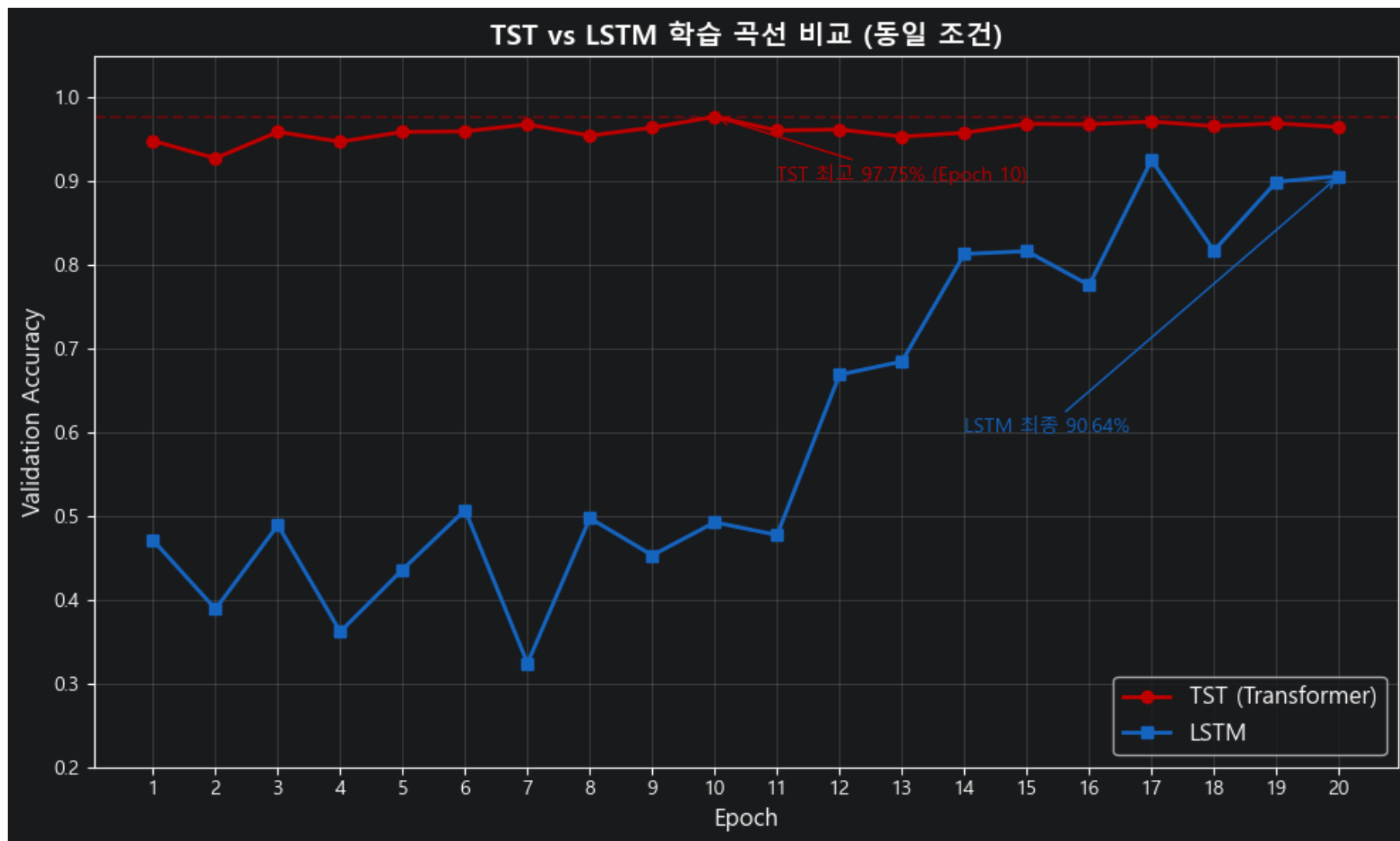
혼동행렬 분석

- ✓ Normal: 503개 전부 정확 (정상/결함 완벽 구분)
- ✓ 주요 오분류: IR→Ball 29 / OR→Ball 19 / Ball→OR 18
(원인: Ball·IR·OR 진동 패턴 유사)

결론

- ✓ 편향 없이 4종 결함 96% 정확 분류

5.3 LSTM vs TST 비교



6. 결론

연구 요약

- Transformer Encoder를 활용해 CWRU 베어링 진동 신호를 4종 (Normal/Ball/IR/OR)으로 분류

주요 결과

- Test 정확도 96%, Macro F1-score 0.96 달성
- Epoch 10에서 Val Acc 97.75%로 최고 성능, 이후 Epoch 부터 과적합 경향 관찰

모델 비교

- 동일 조건에서 TST(97.75%) → LSTM(90.64%)
- Self-Attention의 장거리 의존성 처리 능력이 긴 진동 신호(1024) 분석에 효과적임을 확인

향후 과제

- Early Stopping 적용으로 최적 시점 자동 선택
- 부하 조건별 일반화 성능 검증